

Session 3: Data Types and VariablesTopics:Data types (int, float, char, double).Variable naming rules.Constants, literals.Example: Store and print student's name, age, marks.

- 1) Declare variables for a Flipkart product: productName (as a string), price (float), and rating (double). Assign sample values and print each variable with its data type.**

Code-

```
#include <stdio.h>
```

```
int main() {
```

```
    // Variable declarations
```

```
    char productName[] = "Wireless Earbuds";
```

```
    float price = 1499.99f;
```

```
    double rating = 4.5;
```

```
    // Printing values and data types
```

```
    printf("Product Name: %s\n", productName);
```

```
    printf("Data Type: string (char array)\n\n");
```

```
    printf("Price: %.2f\n", price);
```

```
    printf("Data Type: float\n\n");
```

```
    printf("Rating: %.1f\n", rating);
```

```
    printf("Data Type: double\n");
```

```
    return 0;
```

```
}
```

Output-

```
[Running] cd "c:\Users\Admin\OneDrive\Desktop\practical\" && gcc filpkart.c -o filpkart &&
"c:\Users\Admin\OneDrive\Desktop\practical\"filpkart
Product Name: Wireless Earbuds
Data Type: string (char array)

Price: 1499.99
Data Type: float

Rating: 4.5
Data Type: double
```

- 2) Create a constant variable to store the GST rate (for example, 18%) and use it to calculate the final price of a Zomato order with a given base price.
Constraint: The GST rate must not be changeable after its initial assignment.

Code-

```
#include <stdio.h>
```

```
int main() {
```

```
    const float GST_RATE = 18.0; // Constant GST rate (18%)
```

```
    float basePrice = 500.0;    // Zomato order base price
```

```
    float gstAmount, finalPrice;
```

```
    gstAmount = (basePrice * GST_RATE) / 100;
```

```
    finalPrice = basePrice + gstAmount;
```

```
    printf("Base Price: Rs. %.2f\n", basePrice);
```

```
    printf("GST Rate: %.0f%%\n", GST_RATE);
```

```
    printf("GST Amount: Rs. %.2f\n", gstAmount);
```

```
    printf("Final Price: Rs. %.2f\n", finalPrice);
```

```
    return 0;
}
```

Output-

```
[Done] exited with code=0 in 0.823 seconds

[Running] cd "c:\Users\Admin\OneDrive\Desktop\practical\" && gcc Gst.c -o Gst &&
"c:\Users\Admin\OneDrive\Desktop\practical\"Gst
Base Price: Rs. 500.00
GST Rate: 18%
GST Amount: Rs. 90.00
Final Price: Rs. 590.00
```

3) Write a program that stores your favorite Spotify playlist's name (string), total number of songs (int), and average song duration in minutes (float). Print all values in a single formatted sentence.

Code-

```
#include <stdio.h>
```

```
int main() {
```

```
    char playlistName[] = "Chill Vibes";
```

```
    int totalSongs = 50;
```

```
    float avgDuration = 3.8;
```

```
    printf("My favorite Spotify playlist is '%s', it contains %d songs, and the
average song duration is %.1f minutes.\n",
```

```
        playlistName, totalSongs, avgDuration);
```

```

return 0;

}

```

Output-

```

collect2.exe: error: ld returned 1 exit status
|
[Done] exited with code=1 in 0.61 seconds

[Running] cd "c:\Users\Admin\OneDrive\Desktop\practical\" && gcc spotify.c -o spotify &&
"c:\Users\Admin\OneDrive\Desktop\practical\"spotify
My favorite Spotify playlist is 'Chill Vibes', it contains 50 songs, and the average song duration is 3.8
minutes.

```

4) Identify and correct the invalid variable names from this list:
 1stPlayer, player_age, \$score, total-marks,
 userName.

Hint: Apply standard
 variable naming rules for your programming language.

Variable Name	Valid/Invalid	Reason	Corrected Name
1stPlayer	✗ Invalid	Cannot start with a digit	firstPlayer or player1
player_age	✓ Valid	Uses letters and underscore	player_age
\$score	✗ Invalid	\$ is not a standard character for C identifiers	score
total-marks	✗ Invalid	- is treated as the minus operator	total_marks or totalMarks
userName	✓ Valid	Starts with a letter and uses letters only	userName

Correct Variable Names

```

int firstPlayer;
int player_age;
int score;
int total_marks;

```

```
int userName;
```

Variable Naming Rules in C

1. Must begin with a letter or underscore (_).
2. Cannot begin with a number.
3. Can contain letters, digits, and underscores only.
4. Cannot contain spaces or special characters like -, \$, @, #.
5. Cannot use C keywords (e.g., int, float, return) as variable names.